iSMART

Функции: разбиение программы на части

Как перестать копировать код и начать программировать.





Представьте: вам нужно посчитать площадь круга три раза.

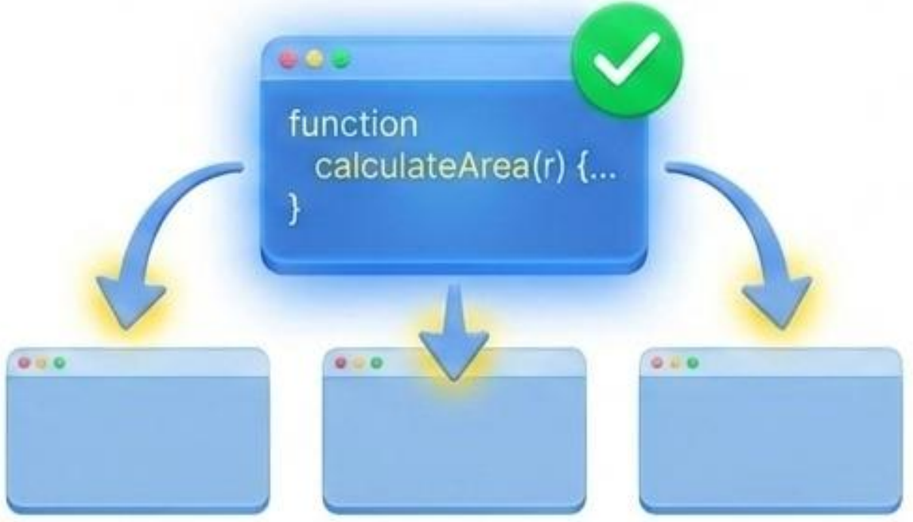
Будете копировать одни и те же строки кода снова и снова? Копирование кода ведет к ошибкам и делает программу огромной и запутанной.

Без функций

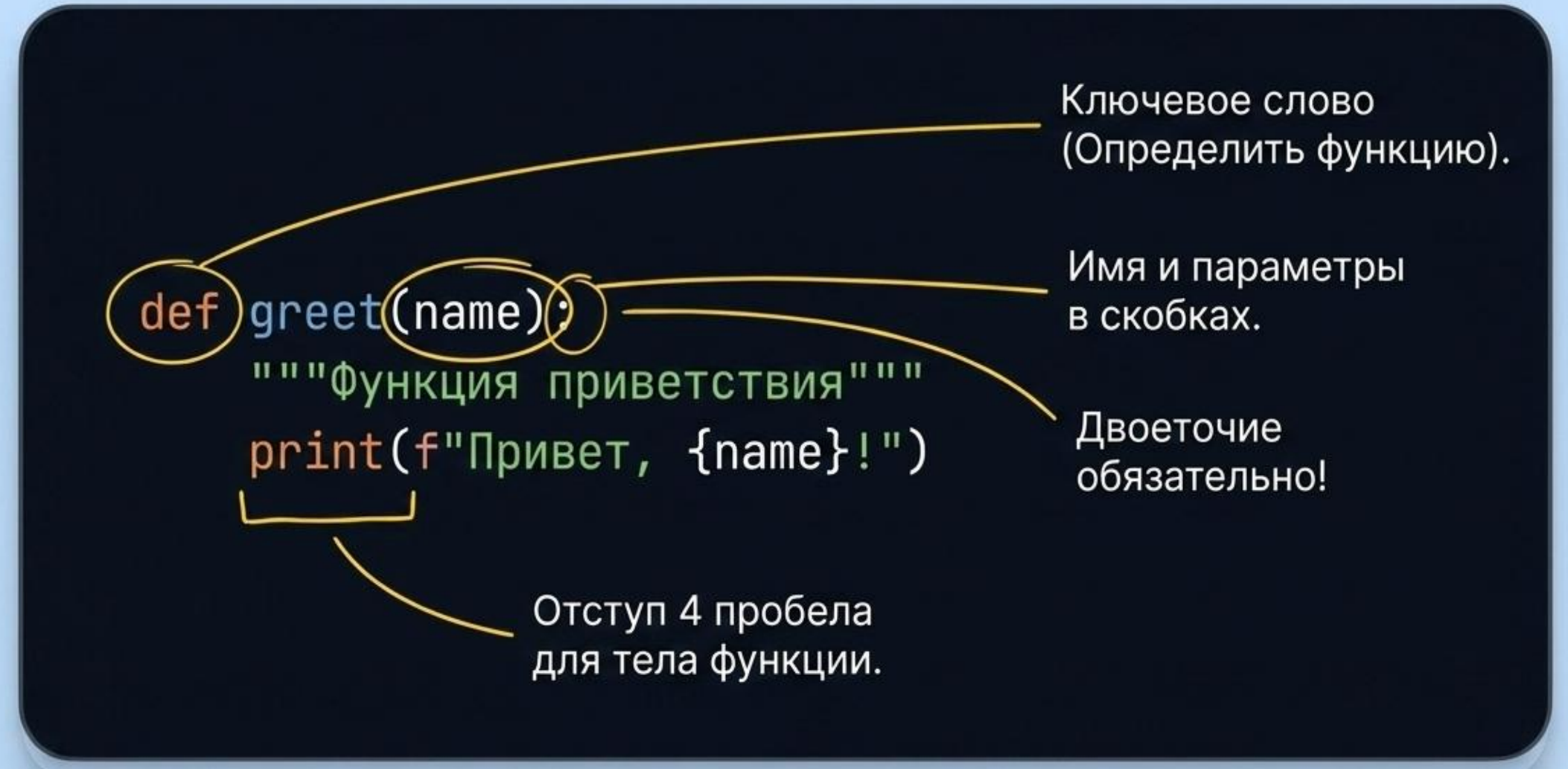


- Код копируется много раз.
- Ошибку нужно исправлять везде.
- Сложно понять логику.

С функциями



- Код написан один раз, вызывается многократно.
- Исправление в одном месте — работает везде.
- Каждая задача имеет четкое имя.


def greet(name):
"""Функция приветствия"""
print(f"Привет, {name}!")
Ключевое слово
(Определить функцию).
Имя и параметры
в скобках.
Двоеточие
обязательно!
Отступ 4 пробела
для тела функции.

Как передать данные внутрь?



Параметр

Переменная в определении функции (пустой слот).

```
def greet(name):
```

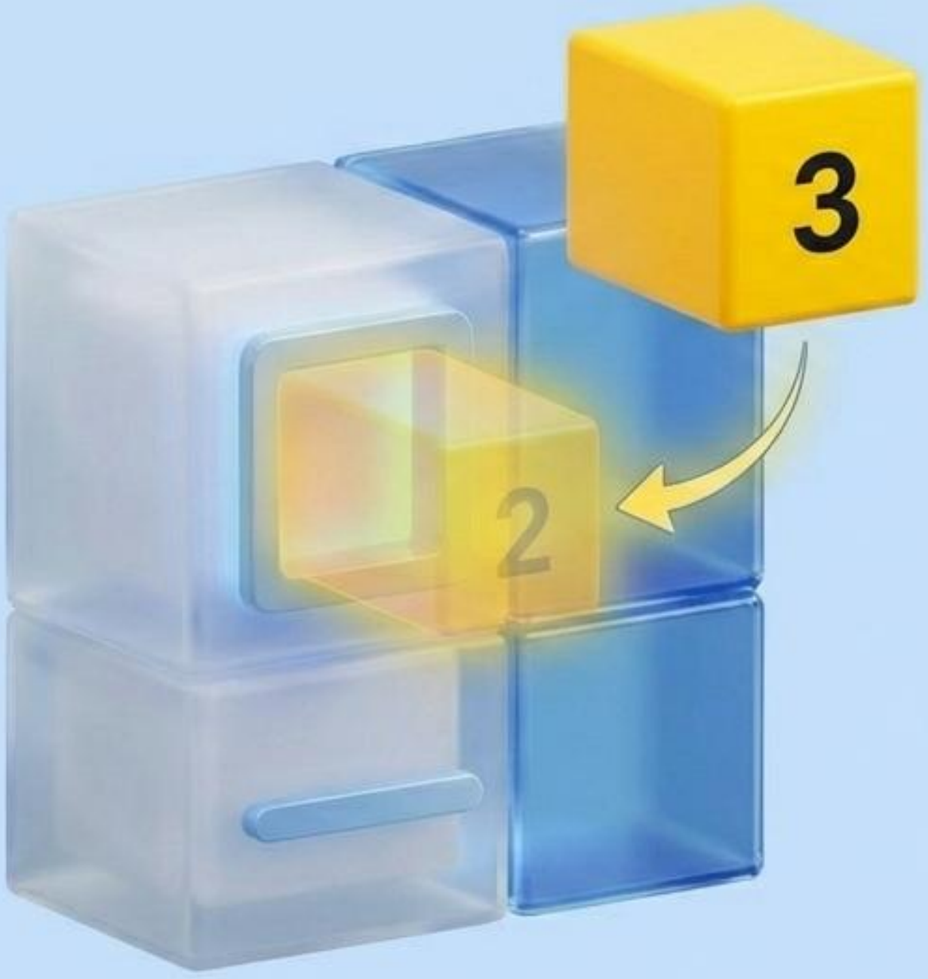
Аргумент

Конкретное значение при вызове (блок, который мы вставляем).

```
greet("Аня")
```

Параметры по умолчанию

Если значение не передано, функция использует значение по умолчанию.

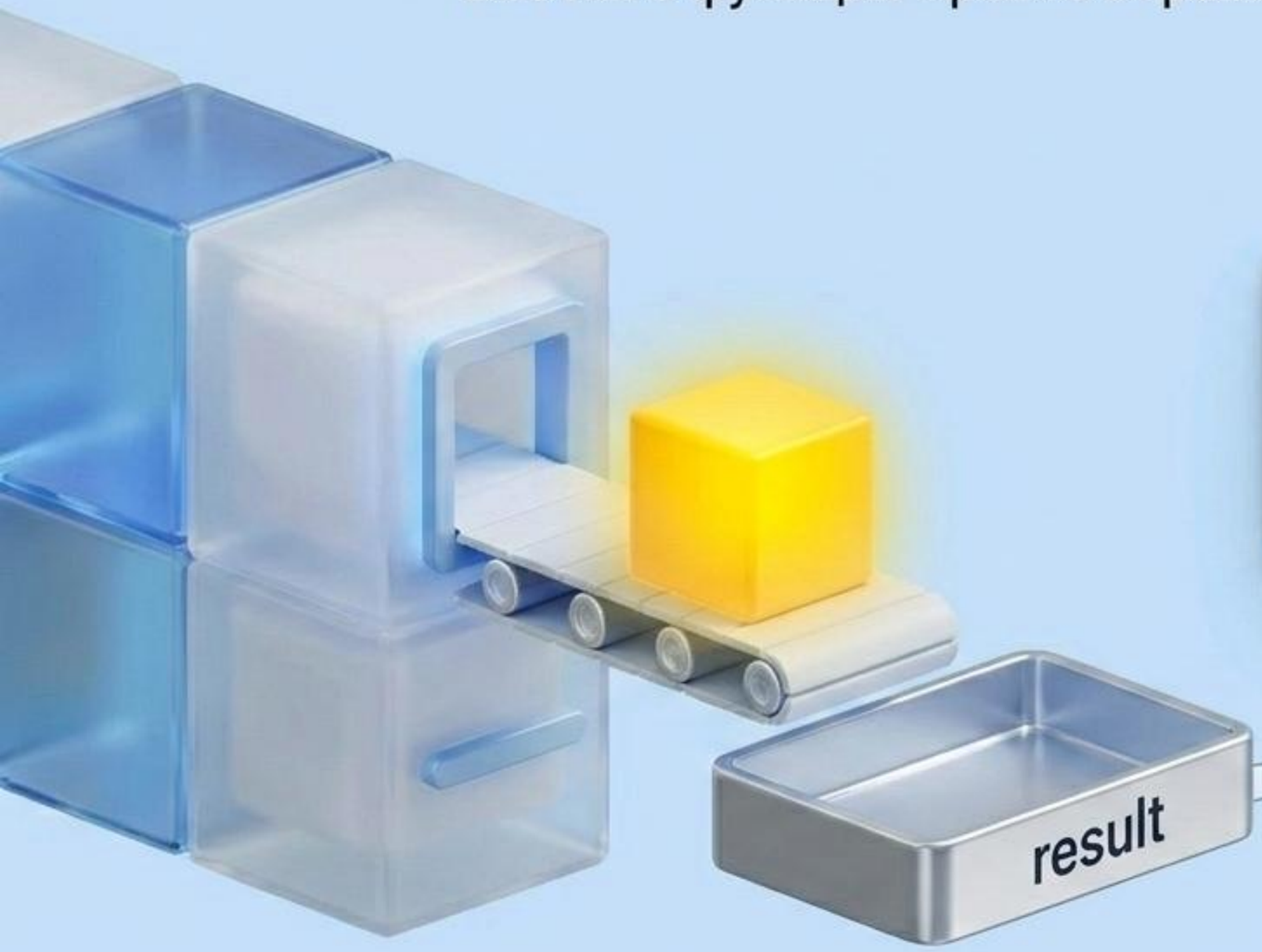



```
def power(base, exp=2):
    return base ** exp

power(3)     # результат 9 (exp = 2)
power(3, 3)  # результат 27 (exp = 3)
```

Оператор return: как получить результат

return завершает функцию и отдает результат обратно в программу.
Без него функция просто отработает, но ничего не вернет.




```
def add(a, b):
    return a + b

result = add(5, 3) # result теперь равен 8
```

Что именно вернет функция?



Нет return

→ Вернет None



return значение

→ Вернет указанное значение



return без значения

→ Вернет None

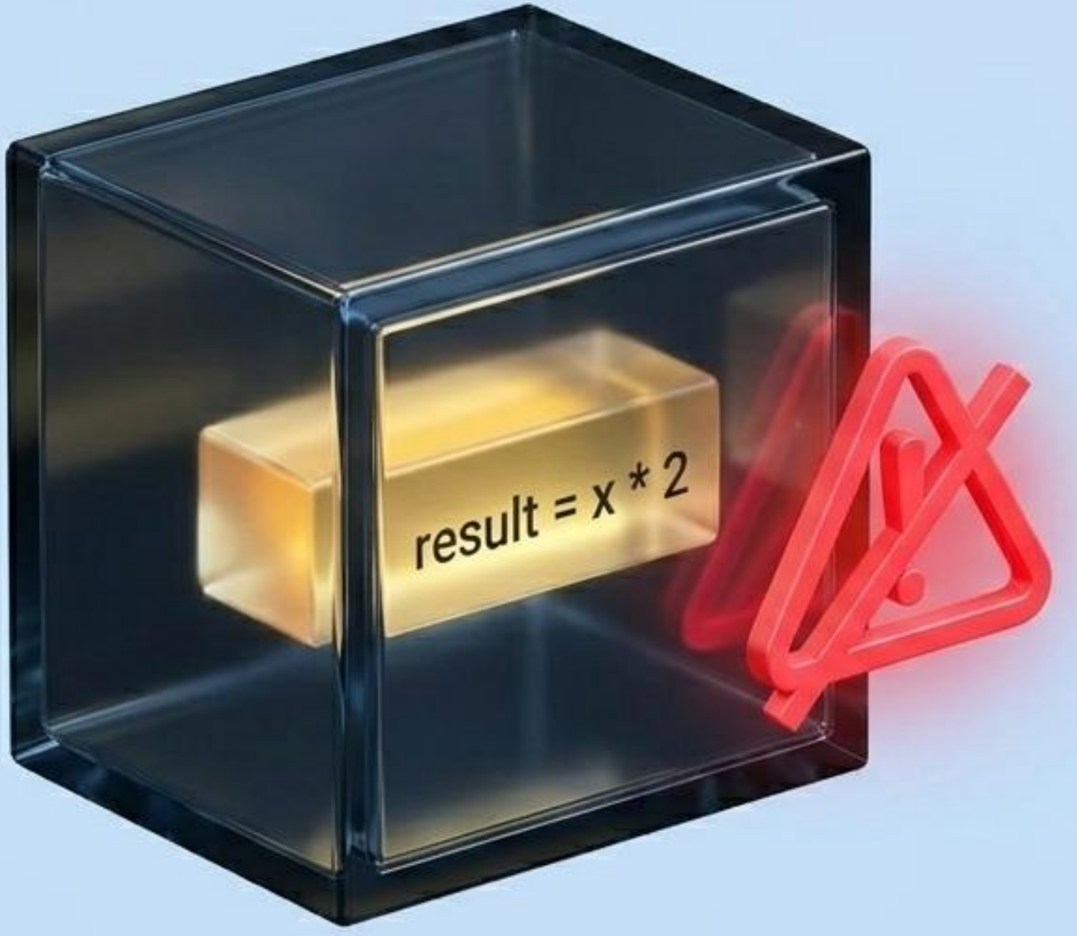


return val1, val2

→ Вернет кортеж (val1, val2)

Правило "Черного ящика" (Локальные переменные)

Переменные, созданные внутри функции, существуют только внутри нее. Снаружи они невидимы!




```
def calculate(x):
    result = x * 2  # Локальная переменная
    return result

print(calculate(5)) # Работает (10)
print(result)       # Ошибка! NameError
```

Функции и обработка данных

Функции — основа обработки датасетов. Мы можем вынести сложную логику классификации в одну функцию и применить ее к тысячам строк.




```
row["class"] = classify(row["score"], row["attendance"], row["project"])
```

Памятка урока

Мини-словарь для закрепления

Если нужно...	Используйте...
Определить функцию	`def name(params):`
Вернуть результат	`return value`
Значение по умолчанию	`def f(x, y=10):`
Вызвать функцию	`result = name(args)`



Текущий контроль

Вопросы для проверки. Отвечайте не заученными правилами, а пониманием логики кода.

ТЕКУЩИЙ КОНТРОЛЬ · ТЕСТ 1

Выберите один правильный ответ

Что вернет функция, если в ней нет оператора return?

А Ошибку (Error)

Б **None**

В Пустую строку ("")

Г Ноль (0)

ТЕКУЩИЙ КОНТРОЛЬ · ТЕСТ 2

Выберите один правильный ответ

В чем разница между параметром и аргументом?

А. Никакой разницы, это одно и то же.

Б. Параметр — переменная при создании (def), аргумент — конкретное значение при вызове.

В. Аргумент — переменная при создании (def), параметр — конкретное значение при вызове.

Г. Аргумент всегда должен быть числом.

Итог урока

- Функция — это главный строительный блок программирования.
- Вы пишете кирпичики кода один раз, а затем собираете из них сложные алгоритмы.
- Все мощные ИИ-библиотеки (pandas, scikit-learn) состоят именно из таких функций.

